

Roll Number:-2070120

Name:-Samiksha Sanjay Adake

Assignment Number:-2

Problem Statement:

Design and implement a C++ program to model a banking system where the class BankAccount keeps track of the total number of accounts created and the total balance across all accounts. Use static data members to maintain these values, and static member functions to retrieve and display them. Add a member function in BankAccount class that takes another BankAccount object as an argument and display the balances in ascending order.

Theory:

1. Static Data Members

Static data members are shared by all objects of a class. Only one copy of a static variable exists, regardless of how many objects are created. In this program, totalAccounts stores the total number of bank accounts created, and totalBalance stores the combined balance of all accounts.

2. Static Member Functions

Static member functions can access only static data members and are called using the class name instead of an object. In

this program, `displayTotalAccounts()` and `displayTotalBalance()` display the total number of accounts and the total balance.

3. Passing Object as a Function Argument

An object of a class can be passed as an argument to another member function. In this program, the function `displayAscending(BankAccount b)` accepts another `BankAccount` object and compares the balances of both accounts.

4. Object Comparison

Objects can be compared by accessing their data members. In this program, the balances of two `BankAccount` objects are compared using the `<` operator to display them in ascending order.

5. Scope Resolution Operator (::)

The scope resolution operator is used to define static data members outside the class and to access static member functions. In this program:

```
int BankAccount::totalAccounts = 0;
```

```
float BankAccount::totalBalance = 0;
```

initialize the static variables, and

```
BankAccount::displayTotalAccounts();
```

```
BankAccount::displayTotalBalance();
```

call the static member functions.

6. Shared Class Data

Shared class data refers to information that belongs to the entire class instead of individual objects. In this program, the total number of accounts and the total balance are common to all objects and are maintained using static data members.

7. Object-to-Object Interaction

One object can interact with another object by passing it as a parameter. In this program, object b1 interacts with object b2 in the `displayAscending()` function to compare their balances.

Code:

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>

class BankAccount
{
private:
    int accNo;
    float balance;

    static int totalAccounts;
    static float totalBalance;

public:
    BankAccount()
    {
        accNo=0;
        balance=0;
    }

    int getData()
    {
        cout<<"\nEnter Account Number (0 to Stop): ";
        cin>>accNo;

        if(accNo==0)
```

```
return 0;
```

```
cout<<"Enter Balance: ";
```

```
cin>>balance;
```

```
totalAccounts++;
```

```
totalBalance+=balance;
```

```
return 1;
```

```
}
```

```
void display()
```

```
{
```

```
    cout<<setw(15)<<accNo
```

```
        <<setw(15)<<balance<<endl;
```

```
}
```

```
float getBalance()
```

```
{
```

```
    return balance;
```

```
}
```

```
void swap(BankAccount &b)
```

```
{
```

```
    BankAccount temp;
```

```
    temp=*this;
```

```
    *this=b;
```

```
    b=temp;
```

```

    }

    static void showTotalAccounts()
    {
        cout<<"\nTotal Accounts Created : "<<totalAccounts;
    }

    static void showTotalBalance()
    {
        cout<<"\nTotal Balance : "<<totalBalance<<endl;
    }

    static int getTotalAccounts()
    {
        return totalAccounts;
    }
};

```

```

int BankAccount::totalAccounts=0;
float BankAccount::totalBalance=0;

```

```

void main()
{
    clrscr();

    BankAccount b[20];

```

```
cout<<"***** BANK ACCOUNT DETAILS *****";
```

```
while(BankAccount::getTotalAccounts()<20)
{
    if(b[BankAccount::getTotalAccounts()].getData()==0)
        break;
}
```

```
cout<<"\n\n===== ACCOUNT DETAILS
===== \n";
```

```
cout<<setw(15)<<"Account No"
    <<setw(15)<<"Balance"<<endl;
```

```
cout<<"-----"<<endl;
```

```
int j;
```

```
for(j=0;j<BankAccount::getTotalAccounts();j++)
{
    b[j].display();
}
```

```
BankAccount::showTotalAccounts();
BankAccount::showTotalBalance();
```

```
int x,y;
```

```
for(x=0;x<BankAccount::getTotalAccounts()-1;x++)
{
    for(y=0;y<BankAccount::getTotalAccounts()-x-1;y++)
    {
        if(b[y].getBalance()>b[y+1].getBalance())
        {
            b[y].swap(b[y+1]);
        }
    }
}
```

```
cout<<"\n\n==== ACCOUNTS SORTED BY BALANCE
(ASCENDING) =====\n";
```

```
cout<<setw(15)<<"Account No"
    <<setw(15)<<"Balance"<<endl;
```

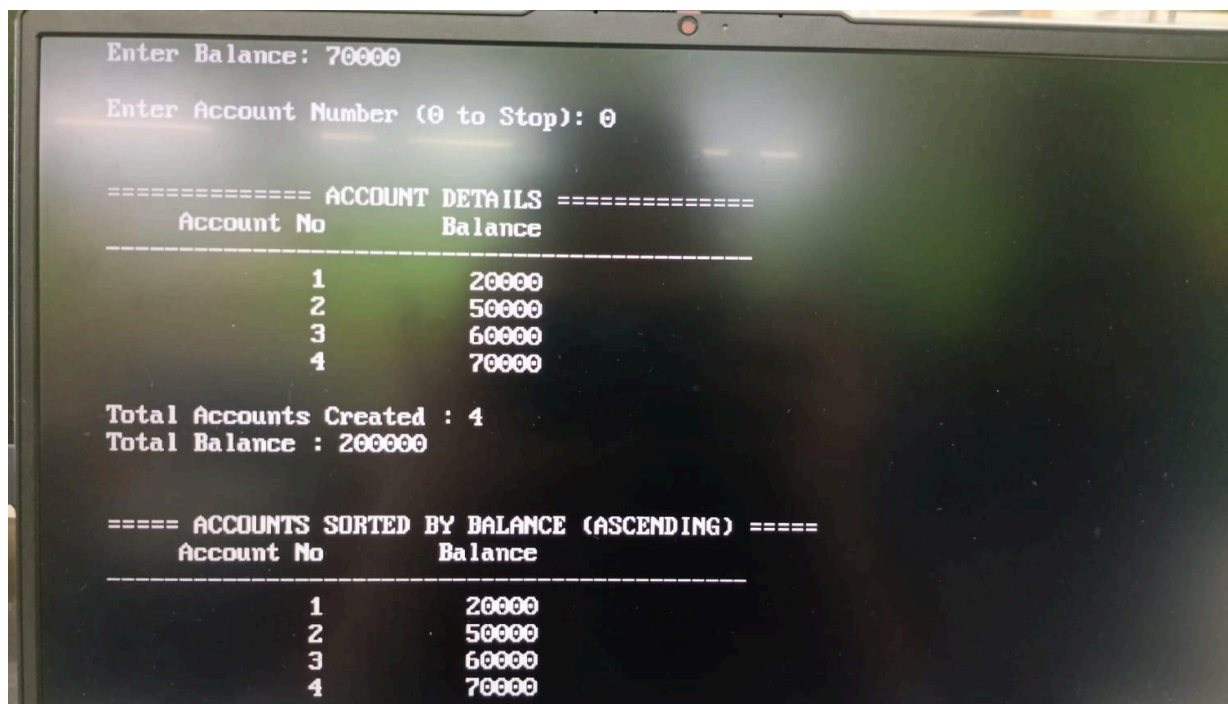
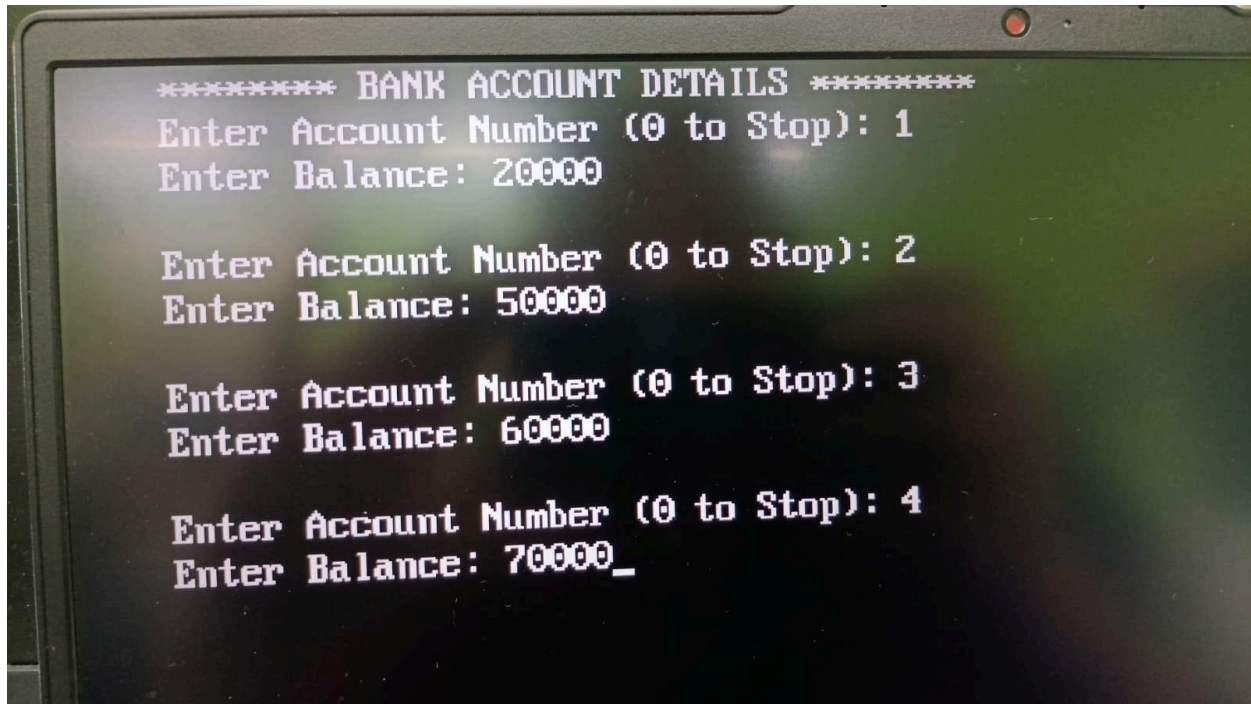
```
cout<<"-----"<<endl;
```

```
for(j=0;j<BankAccount::getTotalAccounts();j++)
{
    b[j].display();
}
```

```
getch();
```


}

Output:



Outcomes:

- Understand the use of static data members and static member functions.
- Learn how to maintain data that is common to all objects.
- Pass an object as a function argument.
- Compare and display account balances in ascending order.
- Apply the concepts of classes, objects, encapsulation, and static members in C++.